

# [ Guía para compilar con librerías externas ]

## Elegir compilador

Existen multitud de compiladores de C++ sobre los que podemos realizar nuestras aplicaciones en OPENGL, además también existen entornos de desarrollo (IDEs) que nos pueden facilitar mucho la creación de la aplicación, a continuación hay una lista de algunos de ellos:

### Gratuitos

- **Visual C++ Express:** Es una versión reducida del Visual Studio especial para estudiantes, cómo entorno es muy potente y cómodo. Para hacerlo funcionar con OpenGL necesitaremos algunas librerías aparte. Podemos descargar la versión 2005 (más estable) de [esta web](#) o la última versión desde [esta otra web](#) pero necesitaremos además descargaros el [Platform SDK](#), para saber cómo instalarlo podemos visitar [esta página](#).
- **GCC:** Es el más sencillo, es libre pero solo es un compilador por lo que puede resultar un poco incómodo, podemos descargaros MinGW que incluye tanto el compilador como las librerías desde la [página oficial](#).
- **Dev-C++:** Entorno de desarrollo gratuito que usa GCC (viene incluido), además trae ya OpenGL y GLUT, [descargarlo de aquí](#).

### De pago (aunque fácilmente localizables...)

- **Visual Studio 6:** Es una versión muy antigua y por lo tanto llena de fallos pero es la más fácil de utilizar, incluye todo lo necesario salvo OpenGL que se tiene que descargar aparte.
- **Visual Studio 2005 o 2008:** Son las versiones profesionales, incluyen todo lo necesario pero a la vez son muy pesadas y con tanta opción pueden hacerse algo complejos.

## Usando GCC bajo Linux

Todas las distribuciones de Linux vienen con GCC instalado, aunque no todas tienen las librerías para desarrollo en OpenGL ni el driver ya instalado. En cualquier caso mirad en vuestro gestor de paquetes e instalar las librerías de desarrollo de todo lo que necesitéis (OpenGL, Glut, GLEW o SDL).

Una vez instalado todo basta estar en la carpeta donde esté el código fuente y poner:

```
gcc *.cpp -lGL -lglut -o test
```

Es importante poner todas las librerías que se vayan a usar en los parámetros del comando, en este caso linkamos contra OpenGL y GLUT, pero si quisiéramos linkar contra SDL pondríamos -lSDL.

## Usando GCC bajo MacOS X

Si queremos usar MacOS X necesitaremos tener instalado GCC, para ello debemos bajarnos el [IDE de desarrollo de Apple \(XCode\)](#) que ya nos instala GCC para que podamos utilizarlo desde la consola.

Luego podemos utilizar lo mismo que si estuviésemos en Linux.

## Librerías para Windows

Es muy común para el programador utilizar librerías externas que realicen funciones muy específicas (sonido, gráficos, etc). Siempre que queramos programar utilizando una librería no estándar (es decir, una

librería de terceros que no venga con nuestro compilador) debemos realizar una serie de pasos que son comunes a cualquier compilador:

## Descargar los headers, libs y binarios

Todas las librerías se suelen componer de tres tipos de archivos, que son:

- **Headers:** Son archivos .h que contienen el prototipo de las funciones, es decir, el nombre y los parametros que necesitan. Mediante los headers el compilador sabe si todas las llamadas a funciones externas son correctas.
- **Libs:** Se conocen como librerías estáticas. Son archivos .lib en VisualStudio y .a en GCC. Son contenedores de funciones ya compiladas. Después de que el compilador haya compilado tu código se hará el linkado, el linker mirará a qué funciones externas llamas desde tu programa y buscará en los libs que tiene que el código compilado de dichas funciones, para juntar tanto tu código compilado como el externo en un solo ejecutable que ya podrá ser utilizado.
- **Binarios:** Llamados librerías dinámicas. En windows son las DLLs. Son contenedores de código ya compilado al igual que las LIBs solo que las DLLs no se usan durante el linkado, sino que se usan durante la ejecución. Si una aplicación usa funciones de una DLL estas no se guardan dentro del ejecutable como pasaba en las LIBs. Lo que se hace es guardar una marca que dice el nombre de la función y en qué DLL está, una vez se ejecute la aplicación el sistema operativo buscará las DLLs que necesite la aplicación y se encargará de enlazarlo todo para que funcione. Esto se hace así para poder compartir código entre diferentes aplicaciones, ahorrando espacio y facilitando las actualizaciones. Si la DLL no la encuentra dará un error indicando que no ha sido encontrada.

## Instalar archivos

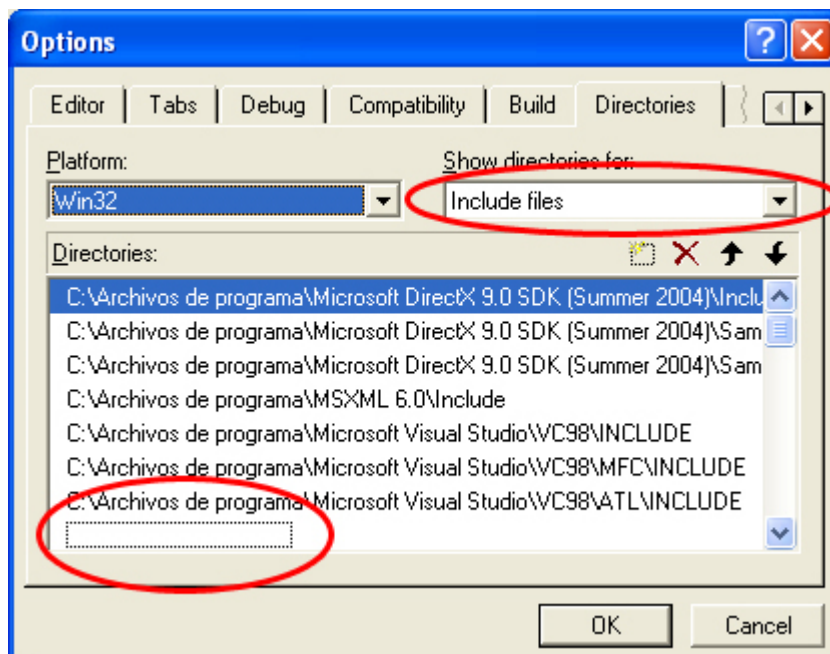
Para que los compiladores puedan usar las librerías es necesario que todos los archivos estén accesibles para él. Eso quiere decir que los headers tienen que estar en el lugar donde él busca los headers, por ello podemos **buscar la carpeta donde el programa guarda los archivos y copiarlos ahí** o si no están allí **decirle las carpetas extra donde deberá buscarlos**.

Pero antes de configurar los programas hay un paso importante que debemos realizar, y es el de copiar las DLLs a algún lugar accesible para el Sistema Operativo, es decir, a una carpeta donde el Sistema Operativo vaya a buscar las DLLs habitualmente, podéis copiar las DLLs a la carpeta **c:\Windows\System32**. Si no hacemos esto aparecerá un error al ejecutar nuestra aplicación indicándonos que no se ha encontrado la DLL necesaria ya que las DLLs se buscan en tiempo de ejecución.

La mayoría de IDEs permite indicarle carpetas extra donde buscar estos archivos, esto basta hacerlo una vez y ya queda accesible para cualquier aplicación que desarrollemos en ese compilador. A continuación hay una lista de pasos en función del programa:

- **Visual Studio 6**

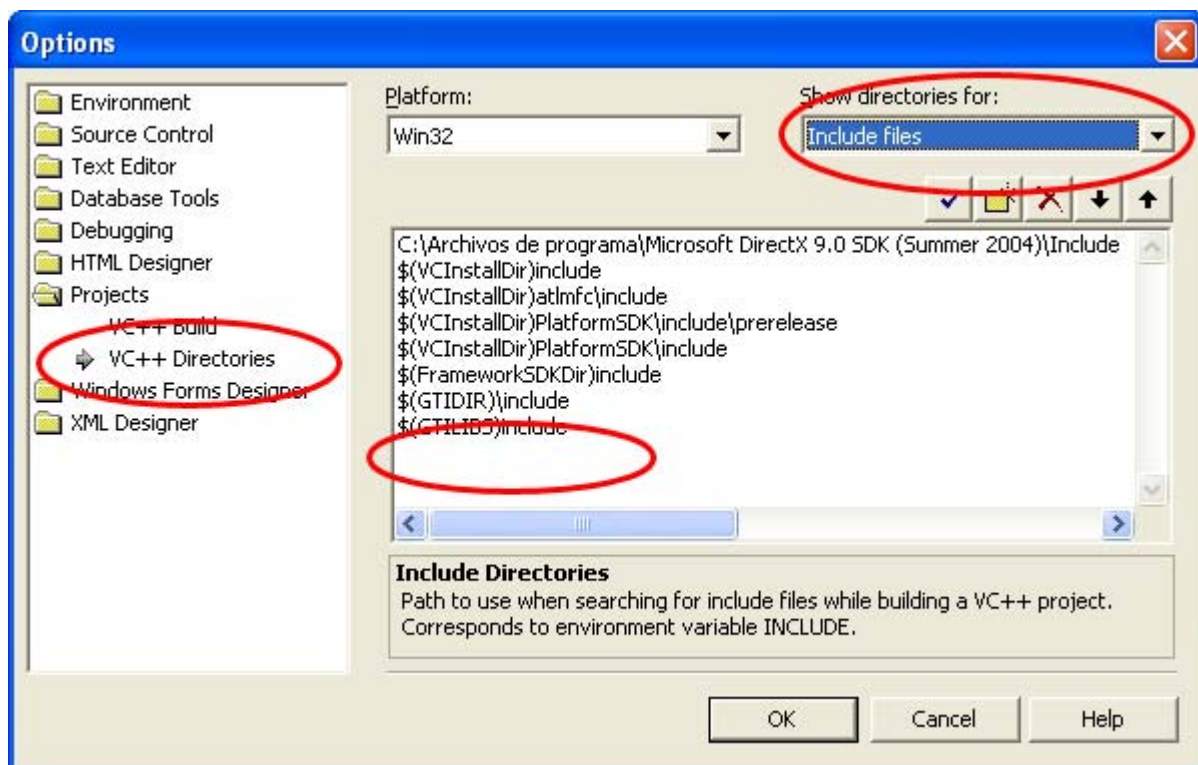
En el menu **Tools->Options** (o **Herramientas->Opciones**) vamos a la pestaña Directories y añadimos la carpeta tanto para los **Include Files** como para los **Library files**. No confundir y poner ambas carpetas en la misma opción.



### • Visual Express

En el menú **Tools->Options** (o **Herramientas->Opciones**) vamos a la carpeta **Projects->VC++ Directories** (o **Proyectos y soluciones->Directorios de VC++**) y añadimos la carpeta con los **includes** en la sección **Includes** y la carpeta con los **libs** en la de **Libraries**. **No confundir y poner ambas carpetas en la misma opción**. Es recomendable poner las librerías para que aparezcan las primeras ya que el orden en la lista es el orden en el que busca los cuerpos de las funciones.

No necesitamos poner una a una todas las librerías, podemos incluir la carpeta raíz que las tiene todas, p.e. **librerías/include**.

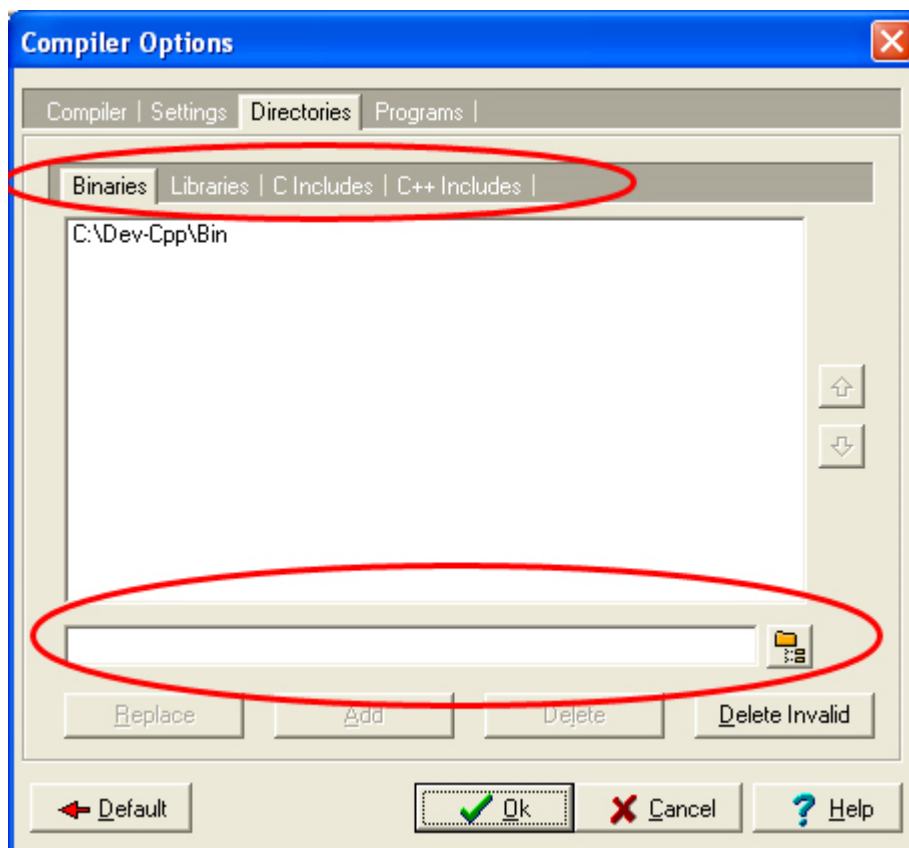


### • Dev-CPP

En el menú **Tools->Compiler Options** vamos a la pestaña **Directories** y añadimos la carpeta tanto para los **C Includes** como para los **Libraries**. No confundir y poner ambas carpetas en la misma

opción.

**Atención: en devcpp no hace falta configurar OpenGL ya que en teoría trae las librerías por defecto.**



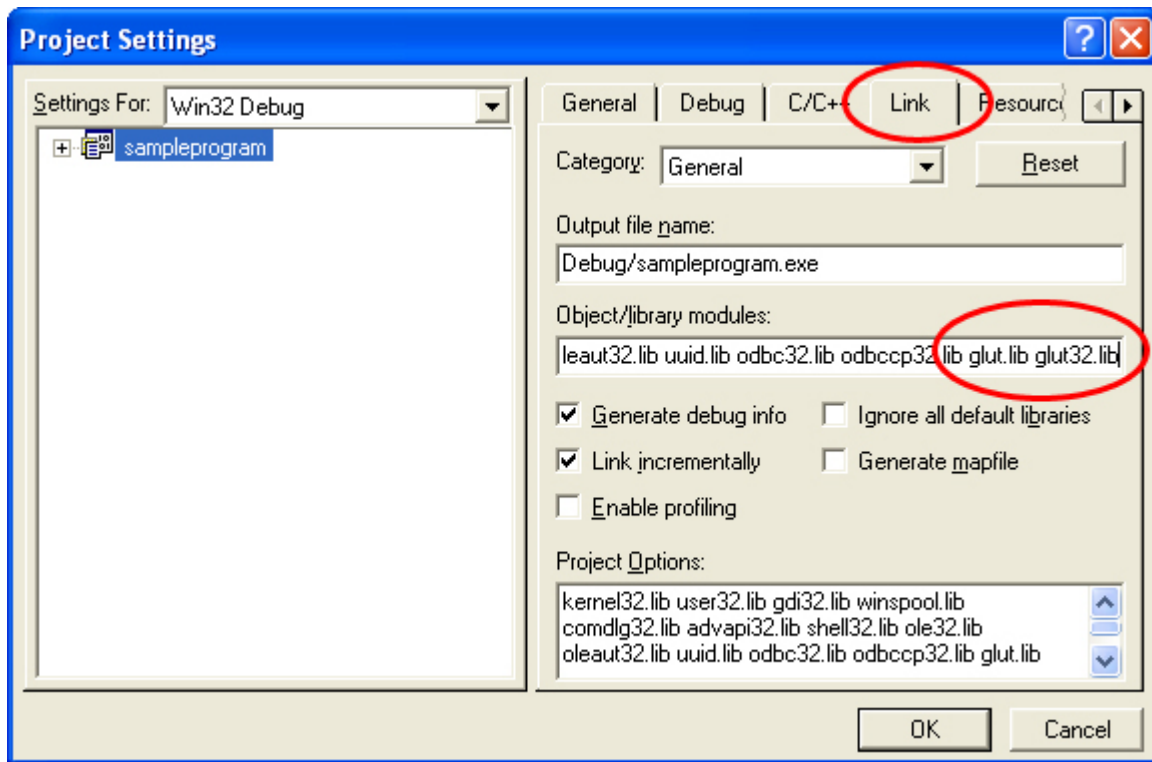
## Configurando

Hay un paso que debemos hacer para cada aplicación que realicemos si utiliza alguna librería externa, es el de decirle contra qué librerías deberá linkar la aplicación, es decir, decirle en qué LIBs deberá buscar las funciones que aparezcan en nuestro programa y que no hayamos programado nosotros. Por defecto él busca en una serie de librerías pero si no encuentra allí dará un error al linkar diciendo que no ha encontrado tal función.

Esto se hace así para evitar que tenga que buscar las funciones en los centenares de archivos LIB que tiene el compilador. En algunos casos este paso no es necesario ya que el archivo header ya le dice al compilador en qué LIB está, pero es importante saber cómo hacerlo:

- **Visual Studio 6**

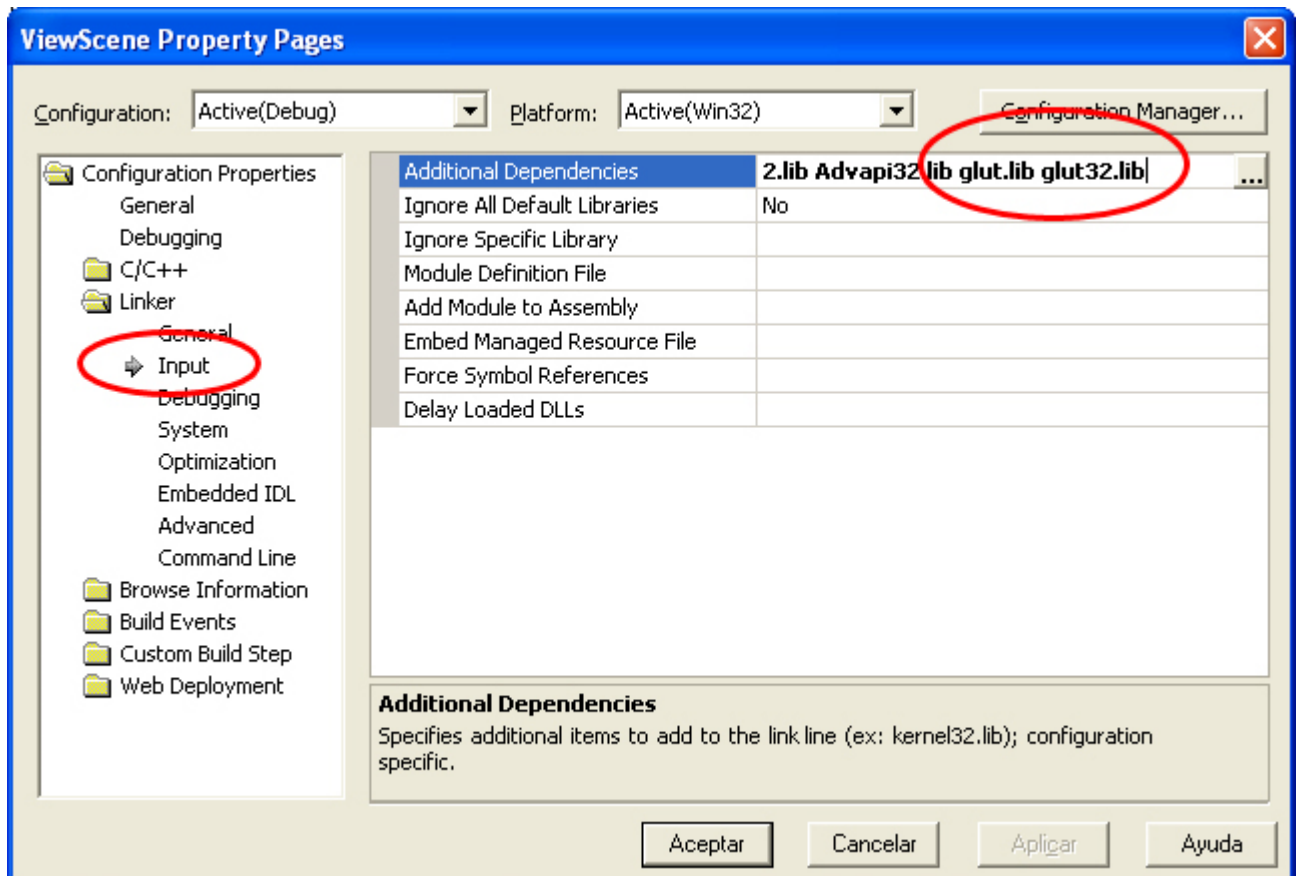
Una vez tenemos creado el proyecto de la aplicación que queremos hacer vamos a **Project->Settings** y pulsamos la pestaña **Link**, y añadimos el nombre de los archivos .LIB que queramos usar, en este caso `opengl32.lib`, `glu32.lib` y `glut32.lib`



### • Visual Express

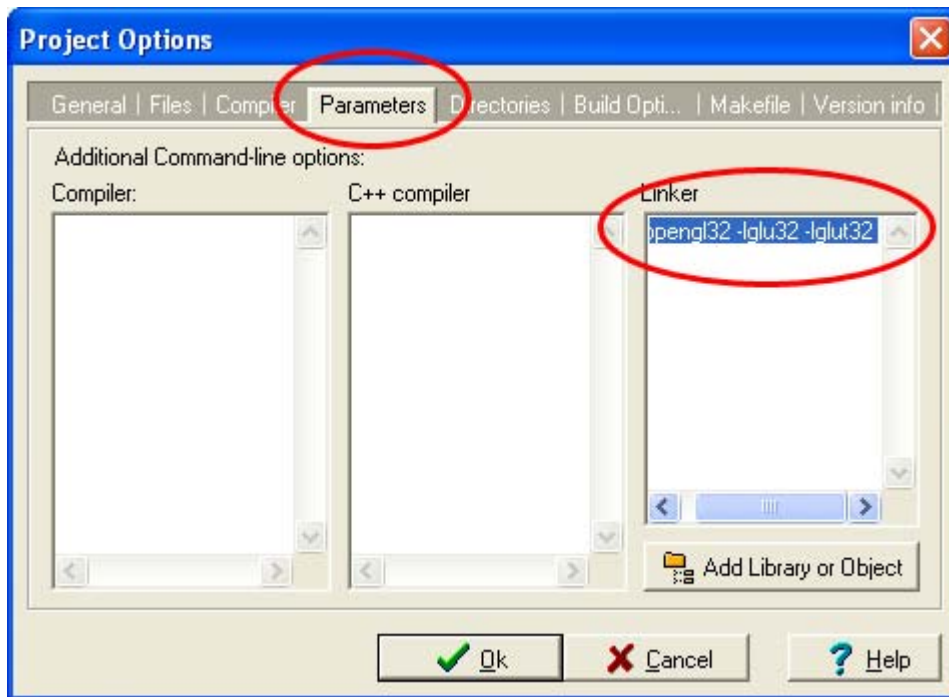
Una vez tenemos creado el proyecto de la aplicación que queremos hacer vamos a **Project->[Nombre] Properties** y navegamos hasta **Configuration Properties->Linker->Input** (o **Propiedades de configuración->Vinculador->Entrada**), y añadimos el nombre de los archivos .LIB que queramos usar en el campo **additional dependencies** (o **dependencias adicionales**).

Si queremos usar OpenGL, GLew y SDL pondremos: **opengl32.lib glu32.lib glut32.lib glew.lib SDLmain.lib SDL.lib**



- **Dev-C++**

Una vez tenemos creado el proyecto de la aplicación que queremos hacer vamos a **Project->Project Options** y pulsamos la pestaña **Parameters**, y añadimos los comandos que indican contra qué librerías hay que linkar, en gcc se pone -l seguido del nombre de la librería, p.e.: **-lopengl32 -lglu32 -lglut32 -lsdl**



## Programando

Si tenemos todo configurado correctamente no debería haber ningún problema, recordad que tenéis que incluir los siguientes archivos en vuestras aplicaciones **por este orden**:

- **windows.h** SOLO si programas desde windows.
- **GL/glut.h**

El windows.h debe estar primero ya que OpenGL sobrescribe partes de las librerías del sistema operativo en windows.

Si tenéis problemas al compilar aseguraos de que los nombres de las funciones son correctos, y de que los includes están bien puestos. Si da el error en el include aseguraos de que los archivos de opengl están en la carpeta GL de donde los hayáis copiado.

Si tenéis problemas al linkar mirad que la configuración del proyecto sea la correcta.

## Errores más comunes

- **He añadido las carpetas de los headers en Include pero da error al hacer include, dice que no encuentra el archivo.**

Revisa bien las inclusiones, a veces añadimos la carpeta **include/GL** y luego hacemos `#include <GL/glut.h>` pero entonces el compilador buscará el archivo en **include/GL/GL/** y no lo encontrará. incluid solo las carpetas **include/** de vuestras librerías.

- **Al linkar me dice que no encuentra la función winmain16**

Eso es porque has creado un proyecto win32 que no es de consola, entonces la aplicación

requiere de un punto de entrada distinto. Crea el proyecto de nuevo y asegurate que especificas que sea de consola.

- **Compila bien pero a la hora de linkar da un error parecido a este: error LNK2001: símbolo externo "public: void \_\_thiscall my\_function(void)" (?myfunction@@@QAEXXZ) sin resolver.**

Esto sucede cuando llamamos a una función y el compilador no encuentra el cuerpo de la función. Si es una función nuestra entonces es que hemos olvidado implementarla, si la función es externa (una función de una librería) entonces el problema puede ser porque no hemos incluido la carpeta de las librerías en la configuración de nuestro compilador, o porque no le hemos dicho que linke contra la librería en cuestión que contiene el cuerpo de esa función. Revisad los pasos descritos anteriormente para asegurarse de que no cometemos ningún error.

- **Compila sin problemas pero al ejecutar salta una ventana que dice que no encuentra una DLL**

Eso se debe porque no has copiado todas las DLLs a algún sitio que pueda encontrar el sistema operativo, revisa los pasos comentados anteriormente.

- **Me sigue diciendo que no encuentra OPENGL.DLL pese a que las tengo copiadas en system32**

La librería OpenGL que instala tu driver se encuentra en OPENGL32.DLL, sin embargo windows tiene una propia llamada OPENGL.DLL que no queremos usar. Si al compilar te insiste en usar la de windows es porque estas linkando contra librerías de opengl de windows, asegurate que linkas con librerías que contienen el número 32 en el nombre (**opengl32.lib, glut32.lib, glu32.lib**).